

One Hot Encoding (OHE)

What is it?

- One Hot Encoding is used to convert **nominal categorical variables** (e.g., colors like Yellow, Blue, Red) into numerical format without introducing **ordinal relationships**.
- Instead of assigning values like Yellow=0, Blue=1, Red=2 (which implies order), OHE creates a **separate column for each category**.

How it works:

- For 3 categories (Yellow, Blue, Red), OHE will create 3 columns: Color_Yellow, Color_Blue, and Color_Red.
- Example encoding:

Yellow → [1, 0, 0]

Blue → [0, 1, 0]

Red → [0, 0, 1]

Why OHE?

- Helps machine learning models **avoid false assumptions of order** between categories.
- Ensures equal importance among nominal values during training.

Dummy Variable Trap

What is it?

- When you use One Hot Encoding, creating **N columns for N categories** can cause **multicollinearity**.
- If the sum of encoded variables is always constant (e.g., always 1), one column can be perfectly predicted using others.

Solution:

- **Drop one column** after One Hot Encoding. Keep only $N - 1$ columns.
 - For example, if you have 3 categories: Yellow, Blue, Red — drop one (usually the first).
- This helps ensure **independence** among features.

Why it matters:

- Models like **Linear Regression** and **Logistic Regression** assume that features are independent.
- Keeping all columns can distort model predictions due to perfect correlation.

Handling High Cardinality in Nominal Data

Problem:

- If a nominal feature (e.g., Brand) has **many unique categories** (like 50+ car brands), OHE will create **too many columns**, increasing dimensionality and slowing down processing.

Practical Solution (Most Frequent Category Encoding):

- Keep only the **most frequent categories** (e.g., top 10 brands).
- Group the remaining less frequent ones into a new category called **"Other"**.
- This reduces the number of columns and simplifies the model without losing major information.

Summary of Best Practices

Topic	Best Practice
Nominal Encoding	Use One Hot Encoding
Dummy Variable Trap	Always drop one OHE column to avoid multicollinearity
High Cardinality Categories	Retain top frequent ones and group others as "Other"
When to use OHE	On nominal data where categories have no inherent order

Topic: One-Hot Encoding in Used Car Dataset (Top Frequent Brands & Pandas get_dummies)

1. Dataset Overview

```
[1]: import pandas as pd
import numpy as np
```

```
[5]: df = pd.read_csv('cars.csv')
df.head()
```

```
[5]:
```

	brand	km_driven	fuel	owner	selling_price
0	Maruti	145500	Diesel	First Owner	450000
1	Skoda	120000	Diesel	Second Owner	370000
2	Honda	140000	Petrol	Third Owner	158000
3	Hyundai	127000	Diesel	First Owner	225000
4	Maruti	120000	Petrol	First Owner	130000

- The dataset contains various columns:
 - Brand
 - Kilometres driven
 - Fuel Type (Diesel, Petrol, CNG, LPG)
 - Owner (First, Second, Third, Fourth, Test Drive Car)
 - Selling Price (target column)

2. Understanding Brand Distribution

- Applied value_counts() on the Brand column

```
[3]: df['brand'].value_counts()
```

```
[3]:
```

brand	
Maruti	2448
Hyundai	1415
Marindra	772
Tata	734
Toyota	488
Honda	467
Ford	397
Chevrolet	230
Renault	228
Volkswagen	186
BMW	120
Skoda	105
Nissan	81
Jaguar	71
Volvo	67
Datsun	65
Mercedes-Benz	54
Fiat	47
Audi	40
Lexus	34
Jeep	31
Mitsubishi	14
Land	6
Force	6
Isuzu	5
Ambassador	4
Kia	4
MG	3
Daewoo	3
Ashok	1
Opel	1
Peugeot	1

Name: count, dtype: int64

- Observations:
 - Some brands appear **very frequently**.
 - Some brands have **very few entries**.
 - Total unique brands: 32

3. Understanding Other Categorical Columns

- Fuel Column:
 - Applied value_counts():

```
[9]: df['fuel'].value_counts()
```

```
[9]:
```

fuel	
Diesel	4402
Petrol	3631
CNG	57
LPG	38

Name: count, dtype: int64

- Categories: Diesel, Petrol, CNG, LPG
 - Two categories dominate.
- Owner Column:
 - Categories: First Owner, Second Owner, Third Owner, Fourth Owner, Test Drive Car
 - 5 unique categories

4. One-Hot Encoding using `pd.get_dummies()`

- One-hot encoding was applied to Fuel and Owner columns:

```
[10]: pd.get_dummies(df, columns = ['fuel', 'owner'])
```

Effect:

- Original Fuel and Owner columns are removed.
- Replaced with:
 - 4 new columns for fuel types (e.g., `Fuel_CNG`, `Fuel_Diesel`, etc.)
 - 5 new columns for owner types (e.g., `Owner_First`, `Owner_Second`, etc.)
- Total columns increased accordingly.

```
[10]:
```

	brand	km_driven	selling_price	fuel_CNG	fuel_Diesel	fuel_LPG	fuel_Petrol	owner_First Owner	owner_Fourth & Above Owner	owner_Second Owner	owner_Test Drive Car	owner_Third Owner
0	Maruti	145500	450000	False	True	False	False	True	False	False	False	False
1	Skoda	120000	370000	False	True	False	False	False	False	True	False	False
2	Honda	140000	158000	False	False	False	True	False	False	False	False	True
3	Hyundai	127000	225000	False	True	False	False	True	False	False	False	False
4	Maruti	120000	130000	False	False	False	True	True	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...
8123	Hyundai	110000	320000	False	False	False	True	True	False	False	False	False
8124	Hyundai	119000	135000	False	True	False	False	False	True	False	False	False

8125 rows × 12 columns

5. Using `drop_first=True` for Dummy Variable Trap

- To avoid multicollinearity, `drop_first=True` is used:

```
[11]: pd.get_dummies(df, columns = ['fuel', 'owner'], drop_first = True)
```

- Drops the first category from each set:
 - For Fuel, if first was CNG → it will be dropped.
 - For Owner, if first was First Owner → it will be dropped.

```
[11]:
```

	brand	km_driven	selling_price	fuel_Diesel	fuel_LPG	fuel_Petrol	owner_Fourth & Above Owner	owner_Second Owner	owner_Test Drive Car	owner_Third Owner
0	Maruti	145500	450000	True	False	False	False	False	False	False
1	Skoda	120000	370000	True	False	False	False	True	False	False
2	Honda	140000	158000	False	False	True	False	False	False	True
3	Hyundai	127000	225000	True	False	False	False	False	False	False
4	Maruti	120000	130000	False	False	True	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...
8123	Hyundai	110000	320000	False	False	True	False	False	False	False
8124	Hyundai	119000	135000	True	False	False	True	False	False	False
8125	Maruti	120000	382000	True	False	False	False	False	False	False
8126	Tata	25000	290000	True	False	False	False	False	False	False
8127	Tata	25000	290000	True	False	False	False	False	False	False

8128 rows × 10 columns

6. Important Notes on Model Usage

- `pd.get_dummies()` is **not** memory-retentive:
 - It **doesn't** remember column order or which dummy was dropped.
 - Not ideal for production use or consistent model input transformation.



Conclusion

- One-hot encoding is simple and effective using pandas get_dummies.
- Use drop_first=True to prevent dummy variable trap.
- For production ML models, prefer OneHotEncoder from scikit-learn over pandas' get_dummies.



One-Hot Encoding using sklearn(OneHotEncoder())

Step 1: Initial Setup

- Start by loading your dataset (e.g., cars data).
- Split your columns into:

```
[7]: from sklearn.model_selection import train_test_split
     x_train, x_test, y_train, y_test = train_test_split(df.iloc[:,4],
                                                    df['selling_price'],
                                                    test_size = 0.2,
                                                    random_state = 0)
```

```
[8]: x_train.head()
```

```
[8]:
```

	brand	km_driven	fuel	owner
3042	Hyundai	60000	LPG	First Owner
1520	Tata	150000	Diesel	Third Owner
2611	Hyundai	110000	Diesel	Second Owner
3544	Mahindra	28000	Diesel	Second Owner
4138	Maruti	15000	Petrol	First Owner

```
[9]: y_train.head()
```

```
[9]:
```

3042	375000
1520	150000
2611	459999
3544	1050000
4138	700000

Name: selling_price, dtype: int64

Step 2: Identify Categorical Columns

- Not all categorical features need encoding.
- Example: Apply encoding **only** on 'fuel' and 'seller_type', but **not** on 'brand' or 'km_driven'.

Step 3: One-Hot Encoding Selected Columns

Use OneHotEncoder from sklearn.preprocessing:

Using OneHotEncoder function

```
[10]: from sklearn.preprocessing import OneHotEncoder

[20]: # ohe = OneHotEncoder()
     ohe = OneHotEncoder(drop = 'first', dtype = np.int32)

[21]: x_train_new = ohe.fit_transform(x_train[['fuel', 'owner']]).toarray()
     x_train_new

[21]: array([[0, 1, 0, ..., 0, 0, 0],
            [1, 0, 0, ..., 0, 0, 1],
            [1, 0, 0, ..., 1, 0, 0],
            ...,
            [0, 0, 1, ..., 1, 0, 0],
            [1, 0, 0, ..., 0, 0, 0],
            [0, 0, 1, ..., 0, 0, 0]], shape=(6582, 7), dtype=int32)

[22]: x_test_new = ohe.transform(x_test[['fuel', 'owner']]).toarray()
```

- drop='first': Drops the first category to avoid multicollinearity.
- sparse=False: Returns a dense NumPy array instead of sparse matrix.

◆ Step 4: Combine Encoded and Non-Encoded Columns

- Extract numeric features or those not being encoded:

```
[24]: new = np.hstack((x_train[['brand', 'km_driven']].values, x_train_new))
```

```
[30]: df_new = pd.DataFrame(new)
      df_new.head()
```

```
[30]:
```

		0	1	2	3	4	5	6	7	8
0	Hyundai	60000	0	1	0	0	0	0	0	0
1	Tata	150000	1	0	0	0	0	0	0	1
2	Hyundai	110000	1	0	0	0	1	0	0	
3	Mahindra	28000	1	0	0	0	1	0	0	
4	Maruti	15000	0	0	1	0	0	0	0	

◆ Step 5: Avoiding Pitfalls

1. ⚠ If you don't use `drop='first'`, you may get extra columns.
2. If you see errors like shape mismatch (e.g., during `np.hstack`), check:
 - Length of input arrays (they must have the same number of rows).
 - Whether encoding is applied on test/train correctly.

◆ Optional Parameters Recap

Parameter	Description
<code>drop='first'</code>	Drops first category to avoid dummy variable trap
<code>sparse=False</code>	Converts sparse matrix to dense array
<code>dtype=float</code>	Forces encoded data to be float type
<code>remainder='passthrough'</code>	Keeps untouched columns in <code>ColumnTransformer</code>

📌 Conclusion:

- Use `.fit_transform()` carefully on training data and `.transform()` on test data.
- Always ensure shapes align before combining arrays with `np.hstack`.

In the dataset, there are multiple categorical columns like `fuel` and `seller_type`, which can be easily encoded using `OneHotEncoder`. However, the **brand** column contains **many categories**, and some of them have **very few occurrences** (less than 100 cars). Handling such **high-cardinality** features is crucial for better model performance.

🌱 Steps for Preprocessing & Encoding on large no. of categories

1. Handling High Cardinality in 'brand'

- The brand column contains **many unique categories**.
- Some brands appear very few times (less than 100 cars), which can cause **sparsity** in one-hot encoding.

2. Strategy to Handle Rare Categories

- Count the number of cars for each brand:

```
[38]: counts = df['brand'].value_counts()
```

Define a threshold (e.g., 10), and **replace brands with fewer occurrences** with a common label like "uncommon":

```
[39]: threshold = 100
      df['brand'].nunique()
```

```
[39]: 32
```

```
[40]: repl = counts[counts < threshold].index
      repl
```

```
[40]: Index(['Nissan', 'Jaguar', 'Volvo', 'Datsun', 'Mercedes-Benz', 'Fiat', 'Audi',
        'Lexus', 'Jeep', 'Mitsubishi', 'Land', 'Force', 'Isuzu', 'Ambassador',
        'Kia', 'MG', 'Daewoo', 'Ashok', 'Opel', 'Peugeot'],
        dtype='object', name='brand')
```

```
[43]: pd.get_dummies(df['brand'].replace(repl,'uncommon'),dtype = np.int32)
```

```
[43]:
```

	BMW	Chevrolet	Ford	Honda	Hyundai	Mahindra	Maruti	Renault	Skoda	Tata	Toyota	Volkswagen	uncommon
0	0	0	0	0	0	0	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	1	0	0	0	0
2	0	0	0	1	0	0	0	0	0	0	0	0	0
3	0	0	0	0	1	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	1	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
8123	0	0	0	0	1	0	0	0	0	0	0	0	0
8124	0	0	0	0	1	0	0	0	0	0	0	0	0
8125	0	0	0	0	0	0	1	0	0	0	0	0	0
8126	0	0	0	0	0	0	0	0	0	1	0	0	0
8127	0	0	0	0	0	0	0	0	0	1	0	0	0

8128 rows x 13 columns

5. Benefits of Grouping Rare Categories

- Reduces the total number of one-hot encoded columns.
- Improves model training time and generalization.
- Avoids overfitting on rarely occurring categories.

Additional Insights

Why Use drop='first' in OneHotEncoder?

- Drops the first column to avoid **dummy variable trap**.
- Reduces multicollinearity.

Sparse Matrix Issue

- By default, OneHotEncoder returns a **sparse matrix**.
- Use sparse_output=False to get a dense NumPy array if needed for concatenation.

Controlling Output Data Types

- After transformation, data types can be controlled using dtype parameter (e.g., dtype=int).

Future Optimization: ColumnTransformer

- The manual extraction + join process is cumbersome.
- Better alternative: ColumnTransformer from sklearn.
 - Apply **different transformers to different columns** in a single line.
 - Cleaner, more maintainable pipelines.
 - This is to be covered in the **next video**.



Conclusion

Handling high-cardinality categorical features like "brand" is essential:

- Combine rare categories into "uncommon".
- Use OneHotEncoder smartly.
- Optimize with ColumnTransformer for larger pipelines.